The method `instance_values` returns a hash that maps instance variable names without "@" to their corresponding values. Keys are strings:

```
class C
  def initialize(x, y)
    @x, @y = x, y
  end
end

C.new(0, 1).instance_values # => {"x" => 0, "y" => 1}
```

Defined in `active_support/core_ext/object/instance_variables.rb`.

2.12.2 `instance_variable_names`

The method `instance_variable_names` returns an array. Each name includes the "@" sign.

```
class C
  def initialize(x, y)
    @x, @y = x, y
  end
end

C.new(0, 1).instance_variable_names # => ["@x", "@y"]
```

Defined in `active_support/core_ext/object/instance_variables.rb`.

2.13 Silencing Warnings and Exceptions

The methods `silence_warnings` and `enable_warnings` change the value of `$VERBOSE` accordingly for the duration of their block, and reset it afterwards:

```
silence_warnings { Object.const_set "RAILS_DEFAULT_LOGGER", logger }
```

Silencing exceptions is also possible with `suppress`. This method receives an arbitrary number of exception classes. If an exception is raised during the execution of the block and is `kind_of?` any of the arguments, `suppress` captures it and returns silently. Otherwise the exception is reraised:

```
# If the user is locked, the increment is lost, no big deal.
suppress(ActiveRecord::StaleObjectError) do
  current_user.increment! :visits
end
```

Defined in `active_support/core_ext/kernel/reporting.rb`.

2.14 `in?`

The predicate `in?` tests if an object is included in another object. An `ArgumentError` exception will be raised if the argument passed does not respond to `include?`.

Examples of `in?`:

```
1.in?([1,2])        # => true
"lo".in?("hello")   # => true
25.in?(30..50)      # => false
1.in?(1)            # => ArgumentError
```

Defined in `active_support/core_ext/object/inclusion.rb`.

3 Extensions to `Module`

3.1 `alias_method_chain`

Using plain Ruby you can wrap methods with other methods, that's called *alias chaining*.

For example, let's say you'd like params to be strings in functional tests, as they are in real requests, but still want the convenience of assigning integers and other kind of values. To accomplish that you could wrap `ActionController::TestCase#process` this way in `test/test_helper.rb`:

```
ActionController::TestCase.class_eval do
  # save a reference to the original process method
  alias_method :original_process, :process
```